

Simulado da prova

Dez metodos novos, no formato exato da prova de amanha. Nenhum repete os dez de ontem, e todos usam so o que voce ja aprendeu. Voce escreve, o programa testa sozinho, e no fim voce clica em Enviar.

quem e voce

senhas fracas

o atalho

a media

em uma linha

o maior nome

quantas contas

ao contrario

o vizinho

a posicao na fila

importar

11 abas · duas rodadas · nenhum assunto novo: hoje e so treino

Os dez do simulado

CADA UM ESTREIA UM PADRAO

#	Metodo	O padrao que estreia
0	MeuNome	o unico sem pegadinha
1	ContarSenhasFracas	contar sem parar no primeiro
2	TelaDoAtalho	switch, case e default
3	MediaTamanhoSenha	somar, dividir, e o zero
4	ListarLogins	o separador vai entre
5	NomeMaisLongo	guardar o melhor ate agora
6	ResumoDaTurma	uma cadeia de if sobre o Count
7	Inverter	o for que anda para tras
8	LoginIgualAoAnterior	comparar com o item anterior
9	PosicaoDoLogin	um while com dois motivos de parar
10	Importar	duas listas ao mesmo tempo

FICA DE FORA



Nao cai amanha

- banco de dados, MySQL e SQL
- { get; set; } - as propriedades
- construtor
- heranca, polimorfismo e encapsulamento
- vetor, ordenacao e recursividade
- a classe Postagem e o mural



Nenhum repete a Aula 12

Os dez de ontem eram pedacos do Conecta. Estes dez sao outros metodos, com os mesmos ingredientes. Se voce fez os de ontem, tem tudo o que precisa.

LEIA ISTO ANTES DE COMECAR

Como o aplicativo funciona

Cada assunto ocupa duas paginas: a primeira explica e mostra um exemplo, a segunda traz o desafio que voce escreve no computador. Leia a primeira, faca a segunda.

O CICLO, E ELE NAO TEM ATALHO

	o que fazer
1	Leia a aba e escreva o metodo em Desafios.cs
2	Salve com Ctrl + S
3	FECHE o programa
4	Rode de novo com F5



O passo 3 nao e opcional

O seu codigo e compilado quando voce roda. Com o programa aberto, editar o arquivo nao muda nada na tela - e o botao Conferir de novo tambem nao.

O QUE MUDA HOJE



As abas ja nascem abertas

Empacou numa? Pule e volte depois. Nenhuma aba esconde a seguinte - numa prova isso seria injusto, e num simulado so atrapalharia.



Nao tem mini-tela

Hoje o que vale sao os testes, e nao brincar com os dados. E assim que a prova de amanha vai ser.

AS CORES DO CODIGO



linha com barra roxa = voce
escreve agora

// ...

cinza e em italico = ja
esta no arquivo, nao
redigite

A prova e amanhã

Sexta-feira, 21/08. São cinco questões práticas no computador e cinco perguntas orais, individuais. As duas metades valem nota.

A PARTE PRÁTICA

Mesmo aplicativo de hoje, com cinco métodos diferentes - da mesma família destes dez. Zero surpresa de mecânica: você vai ter usado o formato três noites seguidas.



O que muda na prova

A explicação e a dica somem da tela. Fica o enunciado, o seu código e os testes. Por isso o que você precisa levar na cabeça está na folha de revisão, no fim desta apostila.



Comece pelo desafio 0

O seu nome, escrito dentro do código. Sem ele o botão Enviar não funciona - e o professor não teria como saber de quem são as respostas.

A PARTE ORAL

Cinco perguntas, uma a uma, enquanto o resto da turma continua a parte prática.



E sobre o SEU código

Não é teoria decorada. As perguntas são sobre o que você acabou de escrever na prova: por que o return está ali, por que o laço é aquele, o que acontece se a lista chegar vazia.



Como treinar hoje

Na correção do projetor, tente explicar em voz alta antes de olhar a resposta. Quem consegue dizer por que o código funciona já respondeu a oral.

P A R T E

0

Antes de tudo

- Quem e voce - o seu nome dentro do codigo

Quem e voce

Antes de qualquer coisa: o professor precisa saber de quem sao estas respostas. E voce escreve isso no proprio codigo, como todo o resto.

Um metodo que devolve texto e a coisa mais simples que existe em C#: ele nao recebe nada, nao percorre nada, nao decide nada. So entrega um valor com return.

Escreva o seu nome completo entre as aspas. Nome e sobrenome - o professor tem trinta alunos e alguns primeiros nomes se repetem.

Voce escreve isto UMA vez. Diferente de uma caixa de texto na tela, o que esta no codigo continua la depois de fechar o programa e rodar de novo com F5.

EXEMPLO

```
public static string Cidade()  
{  
    return "Serra";  
}
```

Sem parametro, sem laço, sem if. So o return com o valor entre aspas.



Guarde esta frase

Um metodo que devolve texto: sem parametro, sem laço, so o return.

VOCE ESCREVE ESTA PARTE

Desafio 0: MeuNome

Devolva o seu nome completo, com nome e sobrenome. E so escrever entre as aspas. Sem este desafio resolvido o botao Enviar nao funciona - o professor nao teria como saber de quem sao as respostas.

O METODO QUE VOCE COMPLETA

```
public static string MeuNome ()  
{  
    // TODO  
}
```



A dica

E uma linha so: o return com o seu nome entre aspas. Nao apague as aspas.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

o seu nome completo, com nome e sobrenome



so o primeiro nome

Nome e sobrenome. Com trinta alunos, so o primeiro nome nao identifica ninguem - e o teste desta aba recusa nome sem sobrenome de proposito.

P A R T E

1

Rodada 1 - o nivel da prova

- Senhas fracas - contar sem parar no primeiro
- O atalho - switch, case e default
- A media - somar, dividir, e o zero
- Em uma linha - o separador vai entre
- O maior nome - guardar o melhor ate agora

1

CONTAR SEM PARAR NO PRIMEIRO

Senhas fracas

Todo laço que voce escreveu ate ontem procurava UMA coisa: achou, saiu na hora. Contar e o contrario disso. Nao existe sair mais cedo, porque a resposta so fica pronta quando a ultima conta ja foi olhada.

Para isso existe o acumulador: uma variavel int que nasce ANTES do laço, valendo zero, e cresce dentro dele. Ela precisa nascer antes por um motivo simples. Se nascesse dentro, cada volta criaria uma variavel nova e zerada, e o total nunca passaria de um.

Somar sobre si mesma e o coracao da coisa. Escrever `fracas = fracas + 1` quer dizer: pegue o valor que fracas tem agora, some um, e guarde de volta em fracas. Nao e equacao de matematica. O sinal de igual aqui e atribuicao, e o lado direito e calculado primeiro.

O return muda de lugar. Nos desafios de busca ele saia de dentro do laço, porque achar uma ja bastava. Aqui ele fica DEPOIS do laço, sozinho, porque o total so esta pronto quando a ultima conta ja foi olhada.

EXEMPLO

```
int atrasados = 0;

foreach (int dias in emprestimos)
{
    if (dias > 30)
    {
        atrasados = atrasados + 1;
    }
}

return atrasados;
```

O total nasce fora, cresce dentro, e sai depois. Aqui o criterio e o prazo de um emprestimo; no seu desafio o criterio e outro.



Guarde esta frase

O contador nasce ANTES do laço, soma sobre ele mesmo dentro, e so e devolvido depois.

1

VOCE ESCRIVE ESTA PARTE

Desafio 1: ContarSenhasFracas

Percorra as contas e devolva QUANTAS tem senha com menos caracteres do que o minimo que chegou. Nao existe achar aqui: o metodo tem que olhar ate a ultima conta antes de responder. Lista vazia devolve zero.

O METODO QUE VOCE COMPLETA

```
public static int ContarSenhasFracas(List<Usuario> contas,
    int minimo)
{
    // TODO
}
```



A dica

Crie a variavel do total ANTES do foreach, valendo zero, e faca ela crescer dentro do laco quando o if for verdadeiro. Para medir a senha use u.Senha.Length, o mesmo Length do porteiro de ontem. O return fica depois do laco, sozinho.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

duas senhas com menos de 4 -> devolve 2

nenhuma fraca -> zero e nao a quantidade de contas

todas fracas -> o total tem que chegar a 3

agora o minimo e 6 -> quem fixou o 4 no codigo erra aqui

senha em branco tem zero caracteres e zero e menos que 4

BORDA lista vazia -> zero e o foreach nao entra nenhuma vez

contador que nao acumula: *Escrever fracas = 1 em vez de fracas = fracas + 1 parece igual e nao e. O total para em um e fica ali, por mais contas fracas que aparecerem depois. O teste com uma conta fraca so passa, e todos os outros denunciam.*

POR QUE TOTAL = 1 NAO E TOTAL = TOTAL + 1

O contador que acumula

Contar e a coisa mais comum que se faz com uma lista, e ela aparece em tres desafios de hoje: o 1, o 6 e o 10. Vale a pena entender uma vez so.

CERTO

```
int fracas = 0;

foreach (Usuario c in contas)
{
    if (c.Senha.Length < minimo)
    {
        fracas = fracas + 1;
    }
}

return fracas;
```

Contar nao para no primeiro: *Diferente de procurar, contar tem que olhar ate a ultima conta. Um return dentro do if aqui devolveria 1 e iria embora.*

ERRADO

```
fracas = 1;
```



O total para em 1

Nao e que ele nao conte: e que ele conta sempre a mesma coisa. Toda volta joga 1 no lugar do que ja estava la. Com tres senhas fracas o metodo devolve 1.



As tres partes

O contador nasce ANTES do laco, soma sobre ele mesmo DENTRO, e e devolvido DEPOIS. Trocar qualquer uma das tres de lugar muda a resposta.

2

SWITCH, CASE E DEFAULT

O atalho

O Conecta tem teclas de atalho, e a pergunta é sempre a mesma: qual tecla o usuário apertou? Quando as opções são valores exatos de uma variável, o switch diz isso mais limpo do que uma fila de if.

O switch compara SO igualdade. Ele não sabe testar maior que, nem menor que, nem juntar duas condições com &&. Para faixas e condições compostas a ferramenta continua sendo o if / else if - foi por isso que na calculadora a divisão por zero ficou no if e a escolha da operação coube no switch.

Os valores comparados aqui são TEXTO, então vão entre aspas: case "F2", e não case F2. Cada case termina com break, que diz: acabou o meu pedaço, pule para depois do switch. Sem o break o projeto nem compila - o compilador para com CS0163.

O default é o case de todo o resto. Ele responde por qualquer tecla que você não escreveu. Atalho sem default é atalho que fica mudo quando o usuário aperta a tecla errada, e o usuário sempre aperta a tecla errada.

Guarde esta frase: switch compara so igualdade. Todo case termina com break, e o default responde por todo o resto.

EXEMPLO

```
string nome = "";

switch (turno)
{
    case "M":
        nome = "Manha";
        break;

    case "N":
        nome = "Noite";
        break;

    default:
        nome = "Turno desconhecido";
        break;
}

return nome;
```

A variável nasce antes do switch, cada case guarda o seu valor, e o return fica depois de tudo, um so.

2

VOCE ESCREVE ESTA PARTE

Desafio 2: TelaDoAtalho

O Conecta recebe o nome da tecla que o usuario apertou e devolve a tela que deve abrir. Use switch: F2 devolve Nova conta, F3 devolve Procurar, F4 devolve Mural e F8 devolve Sair do Conecta. Qualquer outra tecla devolve a frase Atalho sem uso. com o ponto final incluido.

O METODO QUE VOCE COMPLETA

```
public static string TelaDoAtalho(string tecla)
{
    // TODO
}
```



A dica

Sao quatro case e um default. Os valores sao texto, entao vao entre aspas. Cada case termina com break, senao o projeto nem roda. O default nao e enfeite: ele e a resposta para toda tecla que voce nao listou.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

F2 -> a tela de cadastro

F3 -> a busca de contas

F4 -> o mural de recados

F8 -> o ultimo case e nao esqueca dele

tecla que nao esta no switch -> cai no default

BORDA texto vazio tambem cai no default

o default que faltou: Sem o default, a tecla que nao esta em nenhum case nao entra em lugar nenhum: a variavel continua com o valor com que nasceu e o metodo devolve texto vazio. Na tela isso vira um atalho que nao faz nada e nao explica nada. E se faltar um break, nem chega a rodar: o compilador para com CS0163 e a aba inteira fica vermelha.

3

SOMAR, DIVIDIR, E O ZERO

A media

Media e soma dividida pela quantidade. A soma voce acabou de fazer no desafio 1: um acumulador que nasce antes do laço e cresce dentro dele. A quantidade a lista entrega pronta, no Count. O que este metodo cobra e juntar as duas coisas na ordem certa.

E ai aparece a conta que nao pode ser feita. Quando a lista esta vazia o Count vale 0, e dividir por zero nao e conta - e parada. O programa nao avisa antes: ele compila, abre, roda, e morre com DivideByZeroException na hora exata em que chega na barra.

Na calculadora da Aula 9 voce ja barrou uma divisao por zero com um if antes da conta. E a mesma ideia, agora com um motivo a mais: la os numeros eram double e o resultado saia estranho; aqui os numeros sao int e o programa para de vez. O divisor de uma media e sempre a quantidade de itens, e a quantidade de itens as vezes e zero.

Falta um fato novo, e ele e desta noite: dividindo int por int, o C# joga fora o que sobra. 9 dividido por 2 da 4, e nao 4,5. Nao e arredondamento, e corte. Para uma media de caracteres isso serve, e o esperado dos testes ja conta com esse corte.

EXEMPLO

```
if (quantidade == 0)
{
    return 0;
}

return total / quantidade;
```

A divisao por zero se resolve antes de acontecer, nunca depois. Numa media, o divisor e a quantidade de itens.



Guarde esta frase

Media e soma dividida pela quantidade - e quantidade zero para o programa. Confira antes de dividir.

3

VOCE ESCREVE ESTA PARTE

Desafio 3: MediaTamanhoSenha

Some o tamanho de todas as senhas e devolva a media, dividindo pela quantidade de contas. Como o retorno e int, o resto da divisao e descartado. Lista vazia devolve zero - e essa conferencia precisa vir ANTES da divisao.

O METODO QUE VOCE COMPLETA

```
public static int MediaTamanhoSenha(List<Usuario> contas)
{
    // TODO
}
```



A dica

Sao duas coisas em ordem: primeiro barre a lista vazia, depois some e divida. Dividir por contas.Count com a lista vazia nao devolve zero nem devolve resposta errada: para o programa. Para medir cada senha use u.Senha.Length.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

4 e 4 caracteres -> 8 dividido por 2 da 4

5 e 3 -> 8 dividido por 2 da 4

4 e 3 e 2 -> 9 dividido por 3 da 3

9 dividido por 2 da 4 e nao 5: o resto e descartado

uma conta so -> a media e o proprio tamanho

BORDA lista vazia -> zero sem chegar perto da divisao

dividir a lista vazia: Somar zero e dividir por zero parece inofensivo e nao e. Sem o if na frente, a lista vazia derruba o metodo com DivideByZeroException. Todos os outros testes passam, esse trava, e o erro so aparece na hora de rodar.

A lista vazia

Este e o unico desafio de hoje em que o erro nao devolve o numero errado: ele derruba o programa. E o mais importante da rodada 1.

A GUARDA, E ELA VEM ANTES DE TUDO

```
if (contas.Count == 0)
{
    return 0;
}

int soma = 0;

foreach (Usuario c in contas)
{
    soma = soma + c.Senha.Length;
}

return soma / contas.Count;
```

A mesma licao do null: Sempre que voce for usar alguma coisa que pode nao existir - uma divisao pela quantidade, um objeto que pode ser null - a conferencia vem ANTES do uso.

SEM ELA



DivideByZeroException

Somar zero e dividir por zero parece inofensivo, e nao e. Sem o if na frente, a lista vazia derruba o metodo - e o corretor te mostra o nome do erro em vez de um esperado x obtido.



Os outros testes passam

E o que torna esta armadilha traicoeira: com contas na lista o metodo funciona perfeitamente. So o teste da lista vazia reprova. Um teste so, e ele existe por causa disto.

4

O SEPARADOR VAI ENTRE

Em uma linha

Juntar varios pedacos num texto so e o que toda tela de lista faz. O texto comeca vazio e cresce a cada volta: texto = texto + pedaco poe o pedaco no fim do que ja estava la, e nada se perde.

O problema nao e grudar o login. E o separador. Se voce grudar o separador depois de cada login, sobra um pendurado no fim, sem ninguem depois dele. Ontem, no Numerar, o separador ia mesmo em todos e por isso o codigo era simples. Hoje nao vai.

Sao Count logins e Count menos 1 separadores. Tres logins levam dois separadores. Dez levam nove. Sempre um a menos, porque o separador nao acompanha um item: ele fica no espaco ENTRE dois itens, e o ultimo nao tem ninguem depois dele.

A saida e mudar a pergunta. Em vez de perguntar se este e o ultimo - o que o laco nao sabe responder - pergunte se ele e o primeiro. E o proprio texto responde: se ele ainda esta vazio, ninguem entrou antes; se ja tem alguma coisa, o separador cabe.

EXEMPLO

```
string conta = "";

foreach (int n in numeros)
{
    if (conta != "")
    {
        conta = conta + " + ";
    }

    conta = conta + n;
}

return conta;    // "7 + 9 + 8"
```

O separador entra ANTES do item, e so quando ja existe alguma coisa no texto. Repare que ele nunca aparece grudado no fim.



Guarde esta frase

Sao Count itens e Count - 1 separadores: o separador vai ENTRE, nunca depois do ultimo.

4

VOCE ESCREVE ESTA PARTE

Desafio 4: ListarLogins

Monte uma linha unica com os logins das contas, na ordem da lista, separados por espaco barra espaco: os logins ana e bruno viram o texto ana / bruno. Nao pode sobrar separador depois do ultimo nem antes do primeiro. Lista vazia devolve texto vazio.

O METODO QUE VOCE COMPLETA

```
public static string ListarLogins(List<Usuario> contas)
{
    // TODO
}
```



A dica

Comece com um texto vazio e va grudando um login por volta. Antes de grudar, pergunte se o texto ja tem alguma coisa dentro: se tiver, e porque este nao e o primeiro, e ai o separador entra. Compare o texto com aspas vazias. Da para resolver com foreach ou com for - as duas valem.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

dois logins -> um separador so e nenhum no fim

um login sozinho -> separador nenhum

tres logins -> dois separadores

quatro logins -> tres separadores e a ordem e a da lista

BORDA lista vazia -> texto vazio

o separador que sobra no fim: Grudar a barra depois de cada login e o jeito automatico, e e o jeito errado hoje: sobra uma barra solta no fim da linha. Com tres contas quase ninguem percebe. Com uma conta so o erro fica gritante, e o teste de uma conta so existe exatamente para isso.

5

GUARDAR O MELHOR ATE AGORA

O maior nome

Buscar e achar uma conta que voce ja sabe qual e. Escolher o maior e diferente: nenhuma conta sozinha responde a pergunta, porque so da para saber quem e o maior depois de ver todas. Ainda assim o metodo tem que decidir volta a volta, sem guardar a lista inteira.

A solucao e uma variavel que segura o melhor ATE AGORA. Ela comeca valendo null, que aqui quer dizer ainda nao vi ninguem. A cada volta voce compara a conta da vez com esse campeao parcial e, se a nova for melhor, ela toma o lugar. No fim do laco, o melhor ate agora virou simplesmente o melhor.

A primeira volta e o ponto delicado. O campeao ainda e null, e null nao tem Nome nenhum para medir. Por isso a condicao tem duas partes ligadas por `||`: ou o campeao esta vazio, e ai qualquer conta serve, ou o nome da conta da vez e maior. O C# olha o lado esquerdo primeiro e, quando ele ja e verdadeiro, nem chega a olhar o lado direito - e e isso que protege a primeira volta.

Se essa guarda faltar, o programa para com `NullReferenceException` e a tela vai falar em `return null` e em `new` que faltou. Nao e nada disso: o defeito e a guarda ausente. E o empate e decidido pelo sinal que voce escolher - com `>` o primeiro empatado fica, com `>=` o ultimo assume. Sao duas regras diferentes, e o enunciado e que manda.

EXEMPLO

```
int maior = 0;

foreach (int n in numeros)
{
    if (n > maior)
    {
        maior = n;
    }
}

return maior;
```

O campeao nasce antes do laco e so troca quando aparece alguem maior. Aqui ele pode nascer valendo zero. Quando o campeao e um objeto, ele nasce valendo null - e ai o primeiro if precisa tomar cuidado.



Guarde esta frase

Campeao parcial: comeca em null, so troca quando aparece alguem melhor, e empate nao troca.

5

VOCE ESCREVE ESTA PARTE

Desafio 5: NomeMaisLongo

Percorra as contas e devolva aquela cujo NOME tem mais caracteres. Se duas empatarem no tamanho, devolva a primeira das duas. Lista vazia devolve null.

O METODO QUE VOCE COMPLETA

```
public static Usuario NomeMaisLongo(List<Usuario> contas)
{
    // TODO
}
```



A dica

Antes do laço, crie uma variavel Usuario valendo null para segurar o campeao. A cada volta, troque o campeao so se a conta da vez for melhor. E cuidado na primeira volta: nao da para medir o nome de um campeao que ainda nao existe.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

o maior e o SEGUNDO -> quem devolve na primeira volta erra aqui

o maior e o PRIMEIRO -> quem devolve sempre o ultimo erra aqui

o maior esta no meio -> o campeao troca uma vez so

empate de 3 com 3 -> fica a primeira

empate de 4 com 4 no fim -> continua valendo a primeira

BORDA lista vazia -> null e o foreach nao entra nenhuma vez

o campeao começa vazio: *Comparar direto com campeao.Nome.Length na primeira volta estoura o programa com NullReferenceException, porque nesse instante o campeao ainda e null. Nao ha erro de compilacao: o aplicativo abre, roda, e para na primeira conta.*

O melhor ate agora

Achar o maior, o menor, o mais longo: e sempre a mesma forma. Um campeão parcial que comeca vazio e so troca quando aparece alguem melhor.

CERTO

```
Usuario campeao = null;

foreach (Usuario c in contas)
{
    if (campeao == null
        || c.Nome.Length > campeao.Nome.Length)
    {
        campeao = c;
    }
}

return campeao;
```

O empate nao troca: A regra e ficar com o primeiro. Por isso o sinal e > e nao >=: com >=, o segundo de mesmo tamanho tomaria o lugar do primeiro.

OS DOIS ERROS CLASSICOS



Comecar com contas[0]

Funciona em tudo, e quebra so na lista vazia: contas[0] nao existe la. O campeão comeca em null justamente para nao precisar de nenhuma conta para nascer.



Comparar antes de conferir

Se o campeão == null nao vier PRIMEIRO no if, a primeira volta ja estoura com NullReferenceException - o programa tenta ler o Nome de um objeto que nao existe.

P A R T E

2

Rodada 2 - um degrau acima

- Quantas contas - uma cadeia de if sobre o Count
- Ao contrario - o for que anda para tras
- O vizinho - comparar com o item anterior
- A posicao na fila - um while com dois motivos de parar
- Importar - duas listas ao mesmo tempo

6

UMA CADEIA DE IF SOBRE O COUNT

Quantas contas

A barra de status do mural precisa dizer, em uma frase, o tamanho da turma. Não há laço nenhum neste desafio: a lista já sabe quantas contas tem, e quem responde isso é o Count, sem parenteses.

O que existe aqui é decisão. São três frases diferentes para três situações diferentes, e a ferramenta é a cadeia de if que você escreveu no porteiro: o primeiro if que der certo responde, e o método acaba ali mesmo, porque o return sai do método inteiro.

A ordem importa. Comece pelos casos exatos - nenhuma conta, uma conta - e deixe o caso geral por último, sem if nenhum. Se o caso geral vier na frente, ele responde por todos e os outros dois nunca rodam.

E repare na diferença entre zero e um. Zero contas não é um número pequeno de contas: é turma nenhuma, e merece outra frase. Uma conta pede singular. Duas ou mais pedem plural. Quem escreve só a frase do plural mostra 1 conta cadastrada para o primeiro aluno que entrar.

EXEMPLO

```
if (faltas == 0)
{
    return "Nenhuma falta.";
}

if (faltas == 1)
{
    return "1 falta.";
}

return faltas + " faltas.";
```

Os casos exatos vêm primeiro. O caso geral fica por último e não precisa de if - se chegou ali é porque nenhum dos outros serviu.



Guarde esta frase

Count responde quantas. A cadeia de if decide a frase - zero, um e muitos são três casos, não um.

6

VOCE ESCREVE ESTA PARTE

Desafio 6: ResumoDaTurma

Devolva a frase que a barra de status do Conecta mostra. Lista vazia devolve Nenhuma conta cadastrada. Uma conta devolve 1 conta cadastrada. Duas ou mais devolvem o numero seguido de um espaco e da palavra contas cadastradas. As tres frases terminam com ponto final.

O METODO QUE VOCE COMPLETA

```
public static string ResumoDaTurma(List<Usuario> contas)
{
    // TODO
}
```



A dica

Use contas.Count e uma cadeia de if. Nao precisa de laco nenhum. Cuide dos tres casos nesta ordem: zero, um, e todo o resto - e o resto nao precisa de if.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

BORDA lista vazia -> a frase do zero e nao o numero

uma conta so -> singular e sem o s

duas contas -> o numero entra na frase

tres contas -> o Count responde sozinho

quatro contas -> nada de numero fixo no codigo

o plural que serve para tudo: *Devolver so o numero seguido de contas cadastradas parece cobrir os tres casos e nao cobre os dois primeiros: a lista vazia vira 0 contas cadastradas e a primeira conta vira 1 contas cadastradas. Sao os dois testes mais curtos que denunciam.*

7

O FOR QUE ANDA PARA TRAS

Ao contrario

A lista guarda as contas na ordem em que elas chegaram. A tela quase nunca quer essa ordem: em mural, feed e historico o mais novo vai em cima. Ninguem precisa mexer na lista para isso - basta ler ela de tras para frente.

O for tem tres partes e nenhuma delas obriga a andar para frente. Onde comeca vira a ultima posicao. Quanto anda vira `i--`, que tira um a cada volta e e novidade desta noite. E ate quando continua vira uma comparacao invertida: enquanto `i` for maior ou igual a zero.

Os dois numeros das pontas sao os que derrubam a turma. A ultima posicao e `Count - 1`, e nao `Count`, porque `Count` e a quantidade e as posicoes comecam em 0. E a parada e `i >= 0`, nao `i > 0`, porque a posicao 0 e a primeira conta da lista e ela tambem tem que aparecer.

O resto e o de sempre: `contas[i]` com colchetes para pegar a conta daquela posicao, e o texto crescendo com o `+`. Repare que a lista nao muda de ordem em lugar nenhum. Voce mudou so a ordem da leitura.

EXEMPLO

```
string contagem = "";

for (int i = 5; i >= 1; i--)
{
    contagem = contagem + i + "... ";
}

return contagem;    // "5... 4... 3... 2... 1... "
```

Tres partes trocadas de lado: comeca no maior, anda com `i--` e para quando passa do menor. Numa lista, o maior e `Count - 1` e o menor e 0.



Guarde esta frase

De tras para frente: comeca em `Count - 1`, continua enquanto `i >= 0`, e anda com `i--`.

7

VOCE ESCREVE ESTA PARTE

Desafio 7: Inverter

No mural, a conta que chegou por ultimo aparece em cima. Monte o texto com os nomes na ordem inversa da lista, começando pela ultima conta. Cada nome vem seguido de espaco, sinal de maior e espaco - inclusive o ultimo, igual ao Numerar de ontem. Lista vazia devolve texto vazio.

O METODO QUE VOCE COMPLETA

```
public static string Inverter(List<Usuario> contas)
{
    // TODO
}
```



A dica

E um for com as tres partes trocadas de lado: comece na ultima posicao, ande para tras com `i--` e continue enquanto ainda houver posicao valida. A ultima posicao e `Count - 1`, e a posicao 0 ainda faz parte do passeio. Confira o espaco depois do sinal de maior.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

duas contas -> a segunda vem primeiro

tres contas -> a ordem inteira de tras para frente

uma conta so -> ela aparece e aqui o `i > 0` quebra

quatro contas -> nenhuma pode faltar nas pontas

BORDA lista vazia -> texto vazio e o for nao roda nenhuma vez



parar antes da posicao 0

Escrever `i > 0` em vez de `i >= 0` faz o laco parar uma volta cedo demais, e a primeira conta da lista nunca e escrita. Com muitas contas o erro se esconde no meio do texto. Com uma conta so, o metodo devolve nada.

Escolher o laco

Os tres percorrem uma lista. A diferenca nao e o que eles fazem: e o que eles te dao de graca enquanto fazem. Escolher e uma decisao, e nao um habito.

Laco	Quando	Onde cai hoje
foreach	Nao preciso da posicao	Desafios 1, 3, 5, 6
for	Preciso saber em QUE posicao estou	Desafios 7, 8
while	Nao sei quantas voltas vou dar	Desafio 9



O foreach nao sabe onde esta

Ele te entrega o item, e nao o indice. Quando a pergunta e em que POSICAO o item esta, ou qual e o item ANTERIOR, o foreach nao serve - e isso e o desafio 8.

E o passo, no while: *Um while sem a linha que muda a condicao nunca termina. O programa espera 2 segundos, desiste e avisa.*

O FOR QUE ANDA PARA TRAS

```
for (int i = contas.Count - 1;
    i >= 0;
    i = i - 1)
{
    // contas[i]
}
```



Count - 1, e nao Count

Uma lista de 3 tem as posicoes 0, 1 e 2. Comecar em Count ja e uma posicao que nao existe, e o programa estoura na primeira volta.

8

COMPARAR COM O ITEM ANTERIOR

0 vizinho

Ate agora todo laço olhava uma conta por vez e comparava com alguma coisa que veio de fora: um login digitado, um Id procurado. Aqui a comparacao e entre dois itens da propria lista, e isso muda a ferramenta.

O foreach entrega uma conta de cada vez e nao guarda a anterior. Com o for voce tem o i, e com o i voce tem duas contas na mao na mesma volta: `contas[i]` e `contas[i - 1]`. Sempre com colchetes, e sempre lembrando que `contas[0]` e a primeira.

Ai aparece a pergunta que decide o desafio: onde o i comeca? A primeira conta da lista nao tem vizinha de tras. Se o i comecar em 0, a primeira volta ja pede a posicao menos um, que nao existe, e o programa para. Comecando em 1, o primeiro par comparado e o segundo com o primeiro, que e exatamente o que voce quer.

O resto e o desenho de sempre: achou, sai de dentro com o `return`; nao achou, o texto vazio so sai depois que o laço terminar. E repare que a lista vazia e a lista com uma conta so resolvem sozinhas, porque com o i comecando em 1 a condicao ja nasce falsa e o corpo nao roda nenhuma vez.

EXEMPLO

```
for (int i = 1; i < numeros.Count; i++)
{
    if (numeros[i] < numeros[i - 1])
    {
        return "fora de ordem";
    }
}

return "em ordem";
```

O i comeca em 1, e nao em 0, porque a primeira posicao nao tem ninguem atras dela. Este exemplo compara tamanho de numero; o seu desafio compara texto com o sinal de igualdade.



Guarde esta frase

Preciso do item anterior? for com indice comecando em 1 comparando `contas[i]` com `contas[i - 1]`.

8

VOCE ESCREVE ESTA PARTE

Desafio 8: LoginIgualAoAnterior

A lista chegou de uma importacao e pode ter vindo com duas contas SEGUIDAS usando o mesmo login. Percorra com indice e devolva o primeiro login que for igual ao da conta da posicao anterior. Login repetido longe nao vale: so conta quem esta colado. Se nao houver nenhum, devolva texto vazio.

O METODO QUE VOCE COMPLETA

```
public static string LoginIgualAoAnterior(List<Usuario> contas)
{
    // TODO
}
```



A dica

Aqui o foreach nao serve: voce precisa de duas contas ao mesmo tempo, a de agora e a de tras. Com indice isso e `contas[i]` e `contas[i - 1]`. Pense em qual valor o `i` pode ter na primeira volta, porque a primeira conta da lista nao tem ninguem atras dela.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

a segunda repete o login da primeira -> devolve ana

o par repetido esta no fim -> o laco precisa chegar la

o par repetido esta no meio de quatro contas

tres logins iguais -> vale o primeiro par e o return sai do laco

ana repete mas NAO e vizinha -> aqui so vale quem esta colado

uma conta so -> ela nao tem vizinha de tras

BORDA lista vazia -> o for nem comeca

comecar o for em zero: *Com o `i` começando em 0, a primeira volta pede a posicao menos um, que nao existe, e o programa para na hora com erro de indice. Nao e erro de compilacao: compila, roda, e estoura logo no primeiro teste.*

9

UM WHILE COM DOIS MOTIVOS DE PARAR

A posicao na fila

Ontem o while apareceu quando ninguem sabia quantas tentativas seriam necessarias. Hoje ele volta com uma novidade: o laco anda sobre a lista e pode terminar por dois motivos completamente diferentes.

O primeiro motivo e a lista acabar. O segundo e voce ter achado o que procurava. Os dois viram uma condicao so, ligada por &&: continue enquanto ainda tiver lista pela frente E enquanto ainda nao tiver achado. Basta um dos dois falhar para o laco parar.

Esquecer uma das metades da problemas diferentes, e nenhum deles e erro de compilacao. Sem o teste da lista, o indice passa do fim e o programa estoura ao pedir uma posicao que nao existe. Sem o teste de ja achei, o laco continua depois de encontrar e vai guardando as posicoes seguintes por cima, devolvendo a ultima em vez da primeira.

E de novo a diferenca entre posicao e numero na tela. contas[0] e a primeira conta, mas ninguem diz que entrou em zero lugar na fila. Por isso a resposta e `i + 1`, e o 0 fica reservado para dizer nao achei - um valor que nenhuma posicao de verdade usa.

EXEMPLO

```
int i = 0;
int soma = 0;

while (i < precos.Count && soma < 100)
{
    soma = soma + precos[i];
    i = i + 1;
}

return soma;
```

Dois motivos para parar: a lista acabou ou o objetivo foi atingido. Tire a primeira metade e o programa estoura ao passar do fim da lista.



Guarde esta frase

While que percorre lista para por dois motivos: a lista acabou ou eu achei. Os dois no mesmo &&.

9

VOCE ESCREVE ESTA PARTE

Desafio 9: PosicaoDoLogin

Descubra em que lugar da fila esta a conta daquele login, contando a partir de 1: a primeira da lista e a posicao 1. Se o login nao estiver na lista, devolva 0. Use while, e faca o laco parar por dois motivos: a lista acabou, ou voce ja achou.

O METODO QUE VOCE COMPLETA

```
public static int PosicaoDoLogin(List<Usuario> contas,
    string login)
{
    // TODO
}
```



A dica

Antes do laco: um indice começando em 0 e a resposta começando em 0. A condicao do while tem duas partes ligadas por &&. Dentro do corpo, o if guarda a posicao e o indice anda uma casa - e o `i = i + 1` tem que acontecer em TODA volta, senao o laco nunca termina e o aplicativo avisa que o desafio ficou preso.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

a primeira da fila e a posicao 1 e nao 0

a segunda -> o laco precisa dar mais uma volta

a ultima -> o laco anda ate o fim da lista

quatro contas -> a quarta responde 4

duas contas com o mesmo login -> vale a PRIMEIRA e o laco para ali

login que nao existe -> 0 e sem estourar ao passar do fim

BORDA lista vazia -> o while nao entra nenhuma vez

so metade da condicao: Com o while olhando so para ja achei, o indice continua andando quando o login nao existe, passa do fim da lista e o programa para com erro de indice. Com o while olhando so para o tamanho da lista, nao estoura nada, mas a resposta vira a ultima ocorrencia em vez da primeira.

10

DUAS LISTAS AO MESMO TEMPO

Importar

Este é o desafio que junta tudo. Duas listas, um filtro, um contador e a lista crescendo enquanto o laço roda. Cada peça você já usou sozinha; hoje elas aparecem na mesma tela.

Comece separando os papéis. A lista chegando e só leitura: você passa por ela do começo ao fim e não mexe em nada. A lista contas é o destino: é nela que você acrescenta. Confundir as duas troca o sentido do método inteiro.

O filtro decide quem entra. `Ajuda.LoginEmUso` já responde se um login está ocupado - ele não é código seu, já está pronto. A pergunta que você precisa fazer é a contrária: entra quem **NAO** estiver em uso. Escreva `Ajuda.LoginEmUso(contas, u.Login) == false`, que se lê e falso. E repare que a pergunta é sempre sobre a lista contas, que é onde os logins de verdade moram.

E aqui está a virada: a lista contas muda de tamanho a cada conta que entra, e as duas coisas que dependem dela mudam junto. O ID da próxima conta e a quantidade de agora mais um, calculada na hora, dentro do laço. E o próprio filtro fica mais esperto sozinho: se duas contas da mesma remessa pedirem o mesmo login, a primeira entra, e aí o login passa a estar em uso e a segunda é barrada.

Uma tranquilidade: acrescentar em contas enquanto o `foreach` percorre chegando é seguro. O que estourava no desafio Remover de ontem era mexer na **MESMA** lista que o `foreach` estava percorrendo. Aqui são duas listas diferentes.

EXEMPLO

```
int levados = 0;

foreach (Produto p in carrinho)
{
    if (p.Preco < 50)
    {
        sacola.Add(p);
        levados = levados + 1;
    }
}

return levados;
```

Uma lista é percorrida, a outra recebe. Só passa quem o `if` deixa, e o contador anda junto com o `Add`. No seu desafio o filtro não é um preço: é uma pergunta feita à lista de destino.

Guarde esta frase: *Uma lista se percorre, a outra recebe. O Count muda a cada Add, então leia o Count na hora de usar.*

10

VOCE ESCREVE ESTA PARTE

Desafio 10: Importar

Chegou uma remessa de contas novas para entrar no Conecta. Percorra a lista chegando e, para cada conta cujo login ainda nao estiver em uso, de a ela o proximo Id (a quantidade de contas mais um), acrescente na lista contas e conte. Devolva quantas entraram de verdade. Para saber se um login esta ocupado use `Ajuda.LoginEmUso(contas, login)`.

O METODO QUE VOCE COMPLETA

```
public static int Importar(List<Usuario> contas,
    List<Usuario> chegando)
{
    // TODO
}
```

A dica: O laço percorre chegando, mas quem cresce e contas: sao duas listas com papeis diferentes. Para perguntar se NAO esta em uso, compare o resultado com false. E calcule o Id dentro do laço, na hora de usar: a lista muda de tamanho a cada conta que entra.

O QUE O CORRETOR VAI TESTAR

Os testes que o corretor roda

uma chegando com login livre -> entra e recebe o Id 3

o login ana ja esta em uso -> nao entra ninguem

duas entram -> Id 2 e Id 3 e nao o mesmo numero duas vezes

duas chegando com o MESMO login -> so a primeira entra

BORDA lista de destino vazia -> a primeira entra com Id 1

BORDA remessa vazia -> nada entra e nada e alterado

o Id calculado antes do laço: *Guardar contas.Count + 1 numa variavel antes do foreach parece economia e e defeito. O numero congela: a primeira conta da remessa entra com o Id certo e todas as outras entram com o mesmo numero dela. Duas contas com o mesmo Id quebram a busca por Id mais tarde, e o erro so aparece la.*

O botao Enviar

No fim da noite todo mundo clica em Enviar. Nao e formalidade: e o teste de carga da mecanica de amanha, com trinta maquinas e a rede do laboratorio.

O QUE VAI JUNTO

O que	De onde vem
O seu nome	do desafio 0, dentro do codigo
A maquina	para o professor achar o seu lugar
Cada metodo	o codigo que voce escreveu
Os testes	quantos passaram, de quantos
A hora	quando voce clicou



Sem o desafio 0, nao envia

O programa recusa e te leva para a aba 0. Sem o nome, a resposta enviada nao serviria para nada.

SE A REDE FALHAR



O arquivo vem primeiro

Antes de tentar a rede, o programa grava um arquivo na propria maquina. Se a conexao falhar, a sua resposta JA esta salva - e a tela te diz onde.



Rede ruim nao custa prova

Essa e a razao de o arquivo vir antes. Deu erro na tela? Chame o professor: o arquivo esta la, e sobe depois.

Pode enviar de novo: Cada envio e uma linha nova, e nenhum apaga o anterior. Resolveu mais um desafio depois de enviar? Clique outra vez.

A folha de revisao

#	Assunto	Guarde esta frase
0	Quem e voce	Um metodo que devolve texto: sem parametro, sem laco, so o return.
1	Senhas fracas	O contador nasce ANTES do laco, soma sobre ele mesmo dentro, e so e devolvido depois.
2	O atalho	switch compara so igualdade. Todo case termina com break, e o default responde por todo o resto.
3	A media	Media e soma dividida pela quantidade - e quantidade zero para o programa. Confira antes de dividir.
4	Em uma linha	Sao Count itens e Count - 1 separadores: o separador vai ENTRE, nunca depois do ultimo.
5	O maior nome	Campeao parcial: comeca em null, so troca quando aparece alguem melhor, e empate nao troca.
6	Quantas contas	Count responde quantas. A cadeia de if decide a frase - zero, um e muitos sao tres casos, nao um.
7	Ao contrario	De tras para frente: comeca em Count - 1, continua enquanto $i \geq 0$, e anda com $i--$.
8	O vizinho	Preciso do item anterior? for com indice começando em 1 comparando <code>contas[i]</code> com <code>contas[i - 1]</code> .
9	A posicao na fila	While que percorre lista para por dois motivos: a lista acabou ou eu achei. Os dois no mesmo &&.
10	Importar	Uma lista se percorre, a outra recebe. O Count muda a cada Add, entao leia o Count na hora de usar.

O que revisar em casa

A folha de revisao da pagina anterior. Sao onze frases, e cada uma resume um metodo que voce escreveu hoje. Se voce conseguir explicar as onze em voz alta, sem olhar, voce ja respondeu metade da parte oral de amanha.

Os tres erros que derrubam: o contador que nao acumula, a divisao sem conferir a lista vazia, e o campeonato que comeca em `contas[0]`. Os tres passam em quase todos os testes - e e exatamente por isso que eles enganam.

E a decisao do laco. Preciso da posicao? `for`. Nao preciso? `foreach`. Nao sei quantas voltas? `while`. Amanha ninguem vai te perguntar a sintaxe: vao te perguntar por que voce escolheu aquele.